Programmation procédurale

Chapitre 6 : Gestion des erreurs

Automne 2026





Structure du chapitre

1. Introduction
2. Remonter les erreurs
3. Capturer une exception (`try … catch`)
4. `assert`
5. Les garanties d’exception



1. Introduction

Les erreurs de programmation !



- Erreurs durant la phase de développement :
  - Idéalement, une erreur devrait être **détectée** le plus tôt possible : à la **compilation** plutôt qu'à l'exécution.
- Erreurs d'exécution, gestion de l'imprévu :
  - En pratique, vous **ne contrôlez donc pas** le code qui appelle vos fonctions.
  - Vous **ne contrôlez pas** la disponibilité des **ressources** (mémoire, disque, réseau, etc.).
- Il faut **détecter et traiter** les erreurs.

De quelles erreurs parle-t-on ?



Une erreur est ce qui empêche une fonction de produire le résultat escompté.

On en distingue 3 types :

- Erreur de **pré-condition** : typiquement, la fonction reçoit un paramètre non valide.
- Erreur de **post-condition** : il est impossible de retourner le résultat attendu par manque de mémoire, parce que la valeur à renvoyer n'est pas représentable dans le type demandé, etc.
- Erreur **d'invariant de classe**[1] : lorsqu'une méthode change des données membres, et les nouvelles valeurs ne respectent plus les conditions de validité prévues pour la classe (p.ex. une classe Date).

1. Classe (vue au chapitre 8) : un type que l'on peut définir soi-même, qui regroupe des données (données membres) et les fonctions qui les manipulent (méthodes). `string` et `vector` sont des classes de la bibliothèque standard.

Exemple

- Considérons la fonction suivante qui calcule la surface d'un rectangle :

```
int surface(int longueur, int largeur) {
   return longueur * largeur;
}
```

- Il faudrait en préciser :
  - les pré-conditions : la longueur et la largeur doivent être positives
  - la post-condition : la valeur de la surface doit être représentable dans le type `int`

Exemple

- Le code suivant documente et vérifie ces conditions :

```
int surface(int longueur, int largeur)
// pré-conditions : longueur et largeur sont positifs
// post-condition : retourne la surface du rectangle si
//                  elle est représentable
{
   if (longueur <= 0 or largeur <= 0)
      /* signaler l'erreur de pré-condition */;
   unsigned long long s = longueur;
   s *= largeur;
   if (s > numeric_limits<int>::max())
      /* signaler l'erreur de post-condition */;
   return int(s);
}
```



2. Remonter les erreurs

Comment avertir l'appelant ?



Détecter une erreur ne suffit pas, il faut **utiliser cette information**.
En l'état de nos connaissances, nous pourrions :

- Afficher un message à la console d'erreur :

```
if (longueur <= 0 or largeur <= 0)
   cerr << "Erreur de pré-condition";
```

  ce qui aiderait à déboguer, mais pas à gérer des erreurs non fatales.

- Utiliser des valeurs de retour spéciales en cas d'erreur, ce qui avertit le code appelant du problème.

Retourner une valeur spécifique



- Séparer la **valeur calculée** par la fonction et le **code indiquant le succès**.
  Par exemple, passer le résultat par référence et retourner un code d'erreur/succès :

```
const int SUCCESS = 0;
const int PRE_CONDITION_ERROR = -1;
const int POST_CONDITION_ERROR = -2;

int surface(int longueur, int largeur, int& aire) {
   if (longueur <= 0 or largeur <= 0)
      return PRE_CONDITION_ERROR;
   unsigned long long s = longueur; s *= largeur;
   if (s > numeric_limits<int>::max())
      return POST_CONDITION_ERROR;
   aire = int(s);
   return SUCCESS;
}
```



Lever une exception

Ces problèmes sont résolus en C++ par l'utilisation des exceptions.

```
throw expr;
```

où `expr` est une expression de n'importe quel type, sachant que ce type servira à la capturer.

- Mais il existe aussi des **types d'exceptions prédéfinis** qu'il est recommandé d'utiliser.
- Permet **d'interrompre l'exécution normale** d'un programme et de **signaler une erreur** en levant une exception.
- Fonctionnement : une exception levée par **`throw`** est envoyée au code appelant en **remontant la pile d'exécution** jusqu'à ce qu'elle soit **capturée** ou qu'elle **interrompe** le programme (`main`).

Lever une exception



- Reprenons l'exemple de calcul de la surface d'un rectangle en utilisant **`throw`**[1] :

```
#include <stdexcept>
...
int surface(int longueur, int largeur) {
   if (longueur <= 0 or largeur <= 0)
      throw invalid_argument("Longueur et largeur doivent etre positives.");
   unsigned long long s = longueur; s *= largeur;
   if (s > numeric_limits<int>::max())
      throw overflow_error("La surface calculee depasse int::max.");
   return int(s);
}
```

1. `invalid_argument` et `overflow_error` sont des types (des classes) d'exceptions prédéfinis, définis dans `<stdexcept>`.



3. Capturer une exception (`try … catch`)



try { } catch () { }

- La syntaxe générale pour capturer une exception est la suivante :

```
try
{
   // bloc d'exécution normale
}
catch (const TypeException& e)
{
   // bloc de gestion d'erreur
}
```

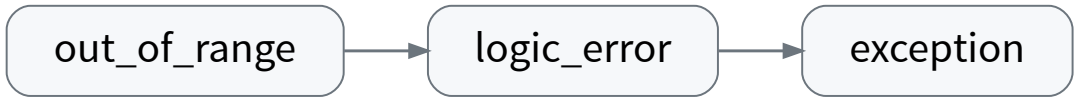
```
try {
   string str("Hello,");
   str.insert(20, " World!");
   cout << str << endl;
}
catch (const std::out_of_range& e) {
   cout << "Out of range" << endl;
}
```

```
Out of range
```

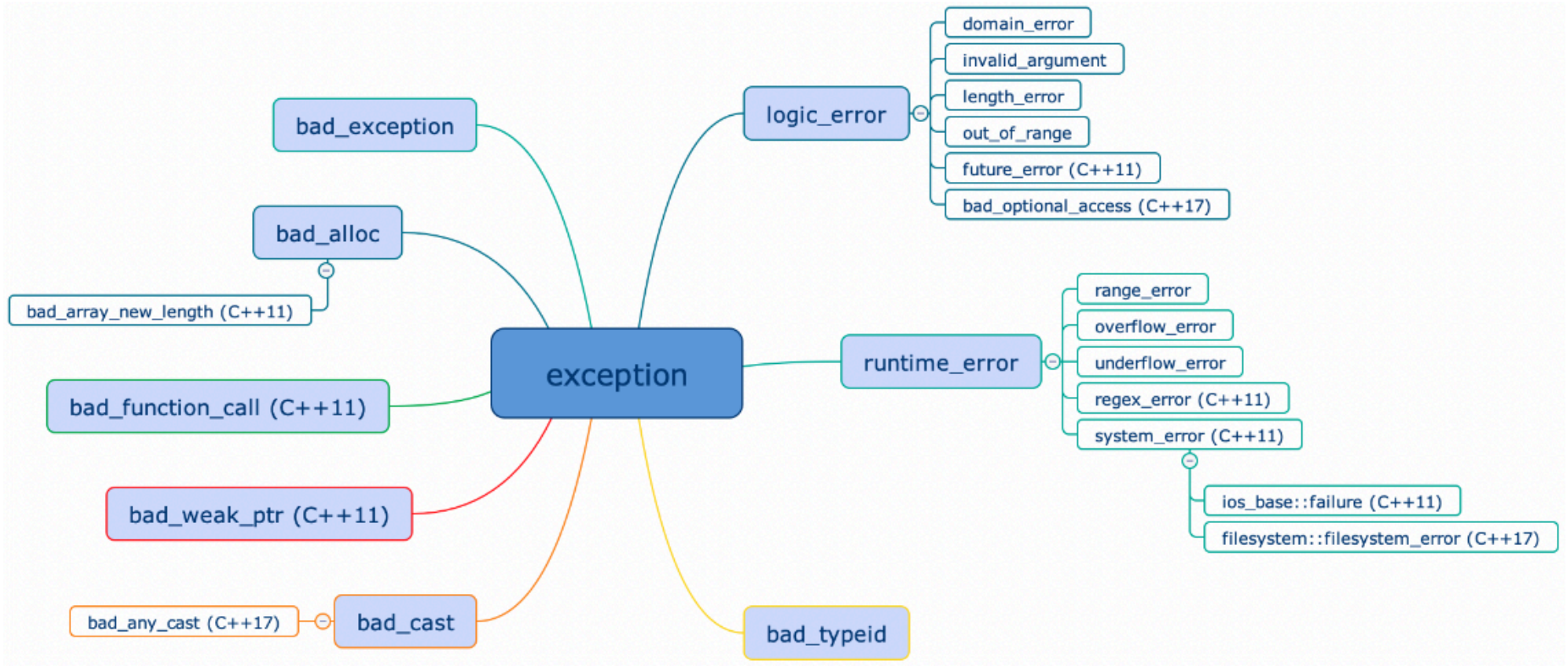
- Il faut spécifier le type d'exception que l'on veut capturer, type que l'on peut trouver dans la documentation des fonctions et méthodes appelées dans le bloc **`try {}`**.
- Seules les exceptions du type `TypeException` **(ou compatibles)** sont capturées par le **`catch`**. Les autres remontent la pile d'exécution, éventuellement jusqu'au `main()`.

Quels types sont capturés ?

- Notons que contrairement au passage de paramètres pour les fonctions, il n'y a **pas de conversion implicite** de type pour le paramètre d'un **`catch`**.
- Seul le type exact est donc capturé, **sauf en présence d'une hiérarchie de classes**.
- Exemple : **`exception`** et **`logic_error`** permettent de capturer l'exception **`out_of_range`** s'il figure comme argument de **`catch`**.



Types d'exceptions prédéfinis



- En-têtes : `exception` dans `<exception>`, `bad_alloc` dans `<new>`, `logic_error`, `runtime_error` et leurs enfants sans mention C++11/C++17 dans `<stdexcept>`. Les autres : voir la liste complète.

Passage par valeur, capture par référence



- Une exception levée est une **copie** de l'expression passée à `throw`. Le `catch` peut la recevoir :
  - par valeur, `catch (std::out_of_range e)` : il en reçoit une nouvelle copie
  - par référence, `catch (const std::out_of_range& e)` : il reçoit l'exception elle-même
- La bonne pratique est de **capturer par référence constante**. Pourquoi ?
  - si le `catch` reçoit par valeur une classe parent (p.ex. `exception`), l'exception est **convertie** du type enfant au type parent : ce qui est propre à l'enfant, comme le message donné à la création de l'exception, est **perdu**

Captures multiples

- Revenons à la documentation de `std::string::insert` :

Exception safety

Strong guarantee: if an exception is thrown, there are no changes in the string.

If `s` does not point to an array long enough, or if either *p* or the range specified by `[first,last)` is not valid, it causes *undefined behavior*.

If *pos* is greater than the string length, or if *subpos* is greater than *str*'s length, an out_of_range exception is thrown.
If the resulting string length would exceed the max_size, a length_error exception is thrown.
A bad_alloc exception is thrown if the function needs to allocate storage and fails.

- Notons que la méthode peut aussi lever des exceptions de type `std::length_error` et `std::bad_alloc` selon l'erreur rencontrée.
- Pour traiter séparément ces erreurs, il faut plusieurs blocs **`catch`** successifs.

Captures multiples

- Un bloc **`try`** peut être suivi de plusieurs blocs **`catch`**.
- L'exception est **capturée par le premier `catch`** qui lui correspond, soit par type exact, soit par héritage.
- Un **seul** des **`catch`** sera **exécuté** !
- Si l'exception levée n'est capturée par aucun **`catch`**, elle continue sa remontée de la pile d'exécution.

```
try {
   // bloc d'exécution normale
}
catch (const std::out_of_range& e) {
   cout << "Out of range" << endl;
}
catch (const std::length_error& e) {
   cout << "Length error" << endl;
}
catch (const std::bad_alloc& e) {
   cout << "Bad alloc" << endl;
}
```

Captures multiples

- Attention, contrairement à la surcharge de fonctions, c'est ici **l'ordre des blocs `catch`** qui compte, pas la « meilleure » correspondance.

```
try {
   string str("Hello,");
   str.insert(20, " World!");
}
catch (const std::out_of_range& e) {
   cout << "Out of range" << endl;
}
catch (const std::exception& e) {
   cout << "Exception std" << endl;
}
```

```
Out of range
```

```
try {
   string str("Hello,");
   str.insert(20, " World!");
}
catch (const std::exception& e) {
   cout << "Exception std" << endl;
}
catch (const std::out_of_range& e) {
   cout << "Out of range" << endl;
}
```

```
Exception std
```

- Certains compilateurs avertissent lorsqu'un **`catch`** en cache ainsi un autre.

catch (...)

- Enfin, il est possible de capturer les exceptions **de tout type** avec la syntaxe **`catch (...)`**.
- Notons que dans le cas de **`catch`** multiples, ce **`catch`** inconditionnel doit être placé en dernier.

```
try {
   throw int(42);
}
catch (const std::exception& e) {
   cout << "Exception standard" << endl;
}
catch (...) {
   cout << "Exception non standard" << endl;
}
```

```
Exception non standard
```



4. assert

assert

- La macro `assert` arrête le programme (`abort`) avec un message d'erreur si `expression` est évaluée à **`false`** :

```
assert(expression)
```

```
#include <cassert>
...
char intToHexa(int value) {
   // arrêt si pas convertible
   assert(value >= 0 and value <= 15);

   if (value < 10)       // '0' – '9'
      return char('0' + value);
   else                  // 'a' – 'f'
      return char('a' + value - 10);
}
```

```
intToHexa(16);
```

```
Assertion failed:
value >= 0 and value <= 15, file
main.cpp, line 33
```

- `intToHexa` **suppose** que l'appelant lui passe une valeur entre 0 et 15 (pré-condition). `assert` vérifie cette supposition pendant le développement : si elle est fausse, le programme s'arrête **tout de suite, à l'endroit du bug**, au lieu de continuer avec un résultat faux.

assert

- Cette macro est utilisée en phase de **développement**, elle peut être **désactivée** si une macro nommée **NDEBUG** a été définie :

```
#define NDEBUG
...
#include <cassert>
```

- `assert` sert donc à **détecter les erreurs de programmation**, pas à traiter les erreurs prévisibles à l'exécution (saisie invalide, fichier absent, …) : celles-ci doivent être traitées par le programme, puisque `assert` peut être désactivé.



5. Garantir l’état du programme après une exception

Une approche contractuelle



- Il est indispensable d'offrir certaines garanties lorsqu'on lève une exception.
- **Ces garanties sont une propriété importante des fonctions** que l'on trouve dans la documentation dans la section intitulée « **exception safety** ».
- Quatre niveaux de garantie sont possibles :
  - **Garantie no-throw** = la fonction ne lève jamais d'exception
  - **Garantie forte** = si elle lève une exception, la fonction laisse le programme dans l'état d'avant l'appel à la fonction
  - **Garantie de base** = si elle lève une exception, la fonction laisse les invariants du programme valides et ne laisse pas de ressources fuiter (p.ex. la mémoire)
  - **Pas de garantie** = si elle lève une exception, on ne peut plus rien supposer sur l'état du programme (invariants cassés, ressources perdues) : continuer après le `catch` n'est pas sûr

La garantie no-throw

- La garantie ultime consiste à savoir qu'une fonction ou méthode **ne lève jamais d'exception**.
- Plus exactement, la fonction garantit que toute exception levée est capturée localement et n'est pas re-propagée.
- Exemple : il est indispensable que les **destructeurs**[1] offrent cette garantie.
  - En effet, `throw` appelle le destructeur de toutes les variables locales avant de sortir d'une fonction pour remonter dans la pile d'exécution.
  - Si un destructeur appelé ainsi lève une exception, elle n'est plus capturable et le programme se termine abruptement.

1. Destructeur (vu au chapitre 8) : méthode d'une classe appelée automatiquement quand une variable de ce type disparaît, p.ex. à la fin du bloc où elle est déclarée. Elle libère ce que la variable utilise, p.ex. la mémoire des éléments d'un `vector`.

La garantie forte

- Si votre fonction peut légitimement lever des exceptions, le but est de fournir une **garantie forte** = en cas d'exception, **l'état précédant** l'appel de la fonction est **inchangé**.
- Un exemple typique de la STL est `vector::push_back` :

Exception safety

If no reallocations happen, there are no changes in the container in case of exception (strong guarantee).
If a reallocation happens, the strong guarantee is also given if the type of the elements is either *copyable* or *no-throw moveable*.
Otherwise, the container is guaranteed to end in a valid state (basic guarantee).

- En effet, `push_back` fait des opérations qui peuvent lever une exception : elle doit parfois **réallouer** la mémoire (quand la taille atteint la capacité) et elle doit toujours **copier** la valeur reçue (p.ex. copier un `string` demande de la mémoire).
- Elle prépare donc tout à côté et ne modifie le vecteur (taille, éléments) **qu'une fois que plus rien ne peut échouer**. Si une exception est levée avant, le vecteur est inchangé.

La garantie de base

- Restaurer l'état initial demande parfois un effort prohibitif. Dans ce cas, il est légitime de ne fournir que la **garantie de base** : si elle lève une exception, la fonction laisse les données dans un état **valide** (invariants respectés, pas de fuite de ressources), mais pas forcément dans l'état d'avant l'appel.
- Un exemple typique de la STL est `vector::insert` :

Exception safety

If the operation inserts a single element at the end, and no reallocations happen, there are no changes in the container in case of exception (strong guarantee). In case of reallocations, the strong guarantee is also given in this case if the type of the elements is either *copyable* or *no-throw moveable*.
Otherwise, the container is guaranteed to end in a valid state (basic guarantee).

- En cas d'insertion à l'intérieur du vecteur, il faut décaler les éléments suivants d'une position vers la fin. Chaque déplacement peut lever une exception, et `insert` ne défait pas les déplacements déjà faits.
- Le vecteur reste alors **utilisable** (taille cohérente, on peut le parcourir, le modifier, le vider), mais son **contenu n'est plus connu**.